

Plantilla de Reporte — Proyecto Final de Probabilidad y Estadística

Título del proyecto

Promedios de alumnos de Ingeniería en Computación (ICO) de la FES Aragón

Integrantes del equipo

- López Rivera Sebastian

1. Definición del problema y de la población

Describan: ¿qué representa la población?, por qué es relevante y el contexto.

La población que toma este proyecto son el conjunto de los promedios de los alumnos de la carrera de Ingeniería en Computación (ICO) de la FES Aragón (300 alumnos, equivalente a 300 valores de promedio)

Usando el de los alumnos de la generación 2025 como parte de la recolección de promedios tanto en el **turno matutino como vespertino** usando su promedio actual contando materias reprobadas y aprobadas cursadas hasta el semestre actual promediado (hasta tercer semestre).

2. Generación de la población (programación)

Expliquen cómo generaron los datos, supuestos, funciones usadas y tamaño de población.

El lenguaje de programación que utilice es java

Los datos generados son simulados de forma aleatoria para no hacer una recopilación de datos exacta por alumno teniendo así datos en un rango desde 5 de calificación hasta un máximo de un 10 y con máximo de dos cifras decimales.

Funciones:

```
public static void main(String[] args) {  
  
    int poblacion = 300;  
  
    int muestra50 = (int)(poblacion * 0.50); // 150  
    int muestra3 = (int)(poblacion * 0.03); // 9  
  
    double minimo = 5.0;  
    double maximo = 10.0;
```

```
Random random = new Random();
```

```
double[] promedios = new double[poblacion];
```

- `public static void main(String[] args) {`: Es el punto de entrada del programa. Cuando ejecutas el código, Java empieza por aquí.
- `int poblacion = 300;`: Crea una variable entera que define el total de alumnos o elementos (300 en total).
- `int muestra50 = (int)(poblacion * 0.50);`: Calcula el 50% de la población (150). El `(int)` convierte el resultado decimal a un número entero.
- `int muestra3 = (int)(poblacion * 0.03);`: Calcula el 3% de la población (9 alumnos).
- `double minimo = 5.0;`: Define la calificación mínima posible (5.0).
- `double maximo = 10.0;`: Define la calificación máxima posible (10.0).
- `Random random = new Random();`: Crea un objeto llamado `random` que nos permitirá generar los números aleatorios.
- `double[] promedios = new double[poblacion];`: Crea un arreglo (lista) vacío llamado `promedios` que tiene espacio para guardar las 300 calificaciones decimales.

```
// GENERAR POBLACION
```

```
for (int i = 0; i < poblacion; i++) {  
    double promedio = minimo + (random.nextDouble() * (maximo - minimo));  
    promedio = Math.round(promedio * 100.0) / 100.0;  
    promedios[i] = promedio;  
}
```

- `for (int i = 0; i < poblacion; i++) {`: Inicia un ciclo que se repetirá 300 veces (desde 0 hasta 299).
- `double promedio = minimo + (random.nextDouble() * (maximo - minimo));`: Genera un número decimal aleatorio entre 5.0 y 10.0. `random.nextDouble()` da un valor entre 0 y 1, que se multiplica por el rango (5) y se le suma el mínimo (5).
- `promedio = Math.round(promedio * 100.0) / 100.0;`: Redondea la calificación generada para que tenga solo dos decimales (ej. 7.34).
- `promedios[i] = promedio;`: Guarda esa calificación ya redondeada en la posición actual `i` del arreglo `promedios`.
- `}`: Cierra el ciclo `for`.

```
// MUESTRA 50%
```

```
double[] muestraA = new double[muestra50];  
for (int i = 0; i < muestra50; i++) {
```

```

    int indice = random.nextInt(poblacion);
    muestraA[i] = promedios[indice];
}

```

```

System.out.println("MUESTRA 50% (150 alumnos)");
mostrarEstadisticas(muestraA);

```

- `double[] muestraA = new double[muestra50];`:: Crea un arreglo vacío para guardar las 150 calificaciones de la muestra.
- `for (int i = 0; i < muestra50; i++) {`: Inicia un ciclo que se repetirá 150 veces.
- `int indice = random.nextInt(poblacion);`:: Genera un número entero al azar entre 0 y 299. Este será el "índice" o posición del alumno que elegiremos.
- `muestraA[i] = promedios[indice];`:: Toma la calificación de la población original en la posición índice y la guarda en nuestra muestraA.
- `}`: Cierra el ciclo for.
- `System.out.println("MUESTRA 50% (150 alumnos)");`:: Imprime en la consola el título de la muestra.
- `mostrarEstadisticas(muestraA);`:: Llama al método `mostrarEstadisticas` (que está más abajo) y le envía los 150 datos para que los analice.

```

// MUESTRA 3%
double[] muestraB = new double[muestra3];
for (int i = 0; i < muestra3; i++) {
    int indice = random.nextInt(poblacion);
    muestraB[i] = promedios[indice];
}

```

```

System.out.println("\n");
System.out.println("MUESTRA 3% (9 alumnos)");
mostrarEstadisticas(muestraB);
}

```

1. `double[] muestraB = new double[muestra3];`:: Crea un arreglo para guardar 9 calificaciones.
2. `for (int i = 0; i < muestra3; i++) {`: Inicia un ciclo que se repite 9 veces.
3. `int indice = random.nextInt(poblacion);`:: Elige un número de posición al azar entre 0 y 299.
4. `muestraB[i] = promedios[indice];`:: Copia el dato de esa posición aleatoria a la muestraB.
5. `}`: Cierra el ciclo for.

6. `System.out.println("\n");`: Imprime un salto de línea (espacio en blanco) en la consola para separar visualmente los resultados.
7. `System.out.println("MUESTRA 3% (9 alumnos)");`: Imprime el título de la segunda muestra.
8. `mostrarEstadisticas(muestraB);`: Llama al método `mostrarEstadisticas` pasándole los 9 datos.
9. `}`: Cierra el método `main`.

```
public static void mostrarEstadisticas(double[] muestra) {
```

```
    Arrays.sort(muestra);
```

```
    int n = muestra.length;
```

1. `public static void mostrarEstadisticas(double[] muestra) {`: Declara un nuevo método que recibe un arreglo de números decimales llamado `muestra`.
2. `Arrays.sort(muestra);`: Ordena todos los datos del arreglo de menor a mayor calificación.
3. `int n = muestra.length;`: Guarda en la variable `n` la cantidad de elementos que tiene la muestra (por ejemplo, 150 o 9).

```
// MOSTRAR DATOS
```

```
System.out.println("\nDatos de la muestra (ordenados):");
```

```
for (int i = 0; i < n; i++) {
```

```
    System.out.println("Dato " + (i + 1) + ": " + muestra[i]);
```

```
}
```

1. `System.out.println("\nDatos de la muestra (ordenados):");`: Imprime un encabezado para la lista de datos.
2. `for (int i = 0; i < n; i++) {`: Recorre cada uno de los elementos de la muestra.
3. `System.out.println("Dato " + (i + 1) + ": " + muestra[i]);`: Imprime en pantalla el número de dato (ej. "Dato 1: ") seguido del valor de la calificación.
4. `}`: Cierra el ciclo `for`.

```
// MEDIA
```

```
double suma = 0;
```

```
for (double val : muestra) suma += val;
```

```
double media = suma / n;
```

1. `double suma = 0;`: Crea una variable en 0 para ir acumulando la suma de todos los datos.
2. `for (double val : muestra) suma += val;`: Es un ciclo "for-each" que recorre todos los valores de la muestra y los va sumando a la variable `suma`.

3. `double media = suma / n;` Calcula el promedio dividiendo la suma total entre el número de datos (`n`).

```
// MEDIANA
double mediana;
if (n % 2 == 0) {
    mediana = (muestra[n / 2 - 1] + muestra[n / 2]) / 2;
} else {
    mediana = muestra[n / 2];
}
```

1. `double mediana;` Declara la variable para guardar la mediana.
2. `if (n % 2 == 0) {`: Verifica si la cantidad de datos es un número par (el residuo de dividir entre 2 es cero).
3. `mediana = (muestra[n / 2 - 1] + muestra[n / 2]) / 2;` Si es par, toma los dos datos que están exactamente en el centro y saca su promedio.
4. `} else {`: Si la cantidad de datos es impar...
5. `mediana = muestra[n / 2];` ...simplemente toma el dato que está en la posición central.
6. `}`: Cierra el bloque if-else.

```
// MODA
double moda = muestra[0];
int maxFrecuencia = 1;
int frecuenciaActual = 1;

for (int i = 1; i < n; i++) {
    if (muestra[i] == muestra[i - 1]) {
        frecuenciaActual++;
    } else {
        if (frecuenciaActual > maxFrecuencia) {
            maxFrecuencia = frecuenciaActual;
            moda = muestra[i - 1];
        }
        frecuenciaActual = 1;
    }
}
```

- `double moda = muestra[0];` Asume inicialmente que el primer dato es la moda (el valor que más se repite).
- `int maxFrecuencia = 1;` Variable para recordar el número máximo de veces que se

ha repetido un número.

- `int frecuenciaActual = 1;` Variable para contar cuántas veces se está repitiendo el número que estamos evaluando en el momento.
- `for (int i = 1; i < n; i++) {` Inicia un ciclo empezando desde el segundo dato (índice 1) hasta el final.
- `if (muestra[i] == muestra[i - 1]) {` Compara si el dato actual es igual al dato anterior (como están ordenados, los iguales estarán juntos).
- `frecuenciaActual++;` Si son iguales, suma 1 al contador actual.
- `} else {` Si el dato es diferente al anterior (cambiamos de número)...
- `if (frecuenciaActual > maxFrecuencia) {` Comprueba si la racha del número anterior superó el récord máximo.
- `maxFrecuencia = frecuenciaActual;` Si lo superó, actualiza el récord máximo.
- `moda = muestra[i - 1];` Actualiza la moda guardando ese número que rompió el récord.
- `}` Cierra el `if` de verificación de récord.
- `frecuenciaActual = 1;` Reinicia el contador a 1 para empezar a contar el nuevo número.
- `}` Cierra el `else`.
- `}` Cierra el ciclo `for` que busca la moda.

```
// RANGO
```

```
double rango = muestra[n - 1] - muestra[0];
```

```
// VARIANZA
```

```
double sumaVar = 0;
```

```
for (double val : muestra) {
```

```
    sumaVar += Math.pow(val - media, 2);
```

```
}
```

```
double varianza = sumaVar / n;
```

```
// DESVIACIÓN
```

```
double desviacion = Math.sqrt(varianza);
```

- `double rango = muestra[n - 1] - muestra[0];` Calcula la diferencia entre el dato más grande (el último del arreglo ordenado) y el más pequeño (el primero).
- `double sumaVar = 0;` Variable para acumular la sumatoria necesaria para la varianza.
- `for (double val : muestra) {` Ciclo que recorre todos los valores de la muestra.
- `sumaVar += Math.pow(val - media, 2);` A cada dato le resta la media, eleva ese resultado al cuadrado (usando `Math.pow`) y lo suma al acumulador `sumaVar`.
- `}` Cierra el ciclo.

- `double varianza = sumaVar / n;` Divide la sumatoria total entre el número de datos para obtener la varianza poblacional.
- `double desviacion = Math.sqrt(varianza);` Calcula la desviación estándar sacando la raíz cuadrada (`Math.sqrt`) de la varianza.

```
// REDONDEO
```

```
media = Math.round(media * 100.0) / 100.0;
mediana = Math.round(mediana * 100.0) / 100.0;
moda = Math.round(modas * 100.0) / 100.0;
rango = Math.round(rango * 100.0) / 100.0;
varianza = Math.round(varianza * 100.0) / 100.0;
desviacion = Math.round(desviacion * 100.0) / 100.0;
```

```
// RESULTADOS
```

```
System.out.println("\nResultados:");
System.out.println("Media: " + media);
System.out.println("Mediana: " + mediana);
```

```
if (maxFrecuencia > 1) {
    System.out.println("Moda: " + moda);
} else {
    System.out.println("Moda: No hay");
}
```

```
System.out.println("Rango: " + rango);
System.out.println("Varianza: " + varianza);
System.out.println("Desviación estándar: " + desviacion);
```

```
// HISTOGRAMA
```

```
histograma(muestra);
}
```

- Las 6 líneas siguientes (`media = ...` hasta `desviacion = ...`) toman cada resultado calculado, lo multiplican por 100, lo redondean a entero y lo vuelven a dividir entre 100.0. Este es un truco clásico para limitar un decimal a solo 2 dígitos (ej. convertir 7.54321 en 7.54).
- `System.out.println("\nResultados:");` Imprime el título "Resultados".
- `System.out.println("Media: " + media);` Imprime el valor de la media.
- `System.out.println("Mediana: " + mediana);` Imprime el valor de la mediana.
- `if (maxFrecuencia > 1) {` Comprueba si la frecuencia máxima fue mayor a 1 (para asegurar que un número realmente se repitió).

- `System.out.println("Moda: " + moda);`:: Si se repitió, imprime el valor de la moda.
- `} else {`: Si ningún número se repitió (frecuencia máxima es 1)...
- `System.out.println("Moda: No hay");`:: Imprime que no existe moda.
- `}`: Cierra el bloque if-else.
- `System.out.println("Rango: " + rango);`:: Imprime el rango.
- `System.out.println("Varianza: " + varianza);`:: Imprime la varianza.
- `System.out.println("Desviación estándar: " + desviacion);`:: Imprime la desviación estándar.
- `histograma(muestra);`:: Llama al método final `histograma` pasándole los datos para que grafique.
- `}`: Cierra el método `mostrarEstadisticas`.

// MÉTODO HISTOGRAMA

```
public static void histograma(double[] muestra) {

    int[] frecuencias = new int[5];

    for (double val : muestra) {
        if (val >= 5 && val < 6) frecuencias[0]++;
        else if (val >= 6 && val < 7) frecuencias[1]++;
        else if (val >= 7 && val < 8) frecuencias[2]++;
        else if (val >= 8 && val < 9) frecuencias[3]++;
        else if (val >= 9 && val <= 10) frecuencias[4]++;
    }
}
```

- `public static void histograma(double[] muestra) {`: Declara el método que generará el gráfico visual en consola basándose en los datos de la muestra.
- `int[] frecuencias = new int[5];`:: Crea un arreglo de números enteros con 5 espacios. Cada espacio actuará como una "cubeta" para contar cuántas calificaciones caen en cada rango.
- `for (double val : muestra) {`: Inicia un ciclo para evaluar cada calificación de la muestra.
- `if (val >= 5 && val < 6) frecuencias[0]++;`: Si la calificación está entre 5.0 y 5.99, suma 1 al contador de la primera cubeta.
- `else if (val >= 6 && val < 7) frecuencias[1]++;`: Si está entre 6.0 y 6.99, suma 1 a la segunda cubeta.
- `else if (val >= 7 && val < 8) frecuencias[2]++;`: Si está entre 7.0 y 7.99, suma 1 a la tercera cubeta.
- `else if (val >= 8 && val < 9) frecuencias[3]++;`: Si está entre 8.0 y 8.99, suma 1 a la cuarta cubeta.
- `else if (val >= 9 && val <= 10) frecuencias[4]++;`: Si está entre 9.0 y 10.0 (incluyendo

el 10 cerrado), suma 1 a la quinta cubeta.

- }: Cierra el ciclo de evaluación.

```
System.out.println("\nHISTOGRAMA DE FRECUENCIAS:");
System.out.println("5 - 6 : " + "*".repeat(frecuencias[0]));
System.out.println("6 - 7 : " + "*".repeat(frecuencias[1]));
System.out.println("7 - 8 : " + "*".repeat(frecuencias[2]));
System.out.println("8 - 9 : " + "*".repeat(frecuencias[3]));
System.out.println("9 - 10 : " + "*".repeat(frecuencias[4]));
}
}
```

- System.out.println("\nHISTOGRAMA DE FRECUENCIAS:");: Imprime el título del gráfico en consola.
- Las 5 líneas siguientes (de 5 - 6 : a 9 - 10 :) imprimen las etiquetas de cada rango y usan el comando `"*".repeat(frecuencias[X])`. Esto toma el símbolo de asterisco * y lo imprime tantas veces como datos hayan caído en esa cubeta, generando así una barra horizontal.
- }: Cierra el método histograma.
- }: Cierra la clase principal Estadística terminando por completo el código.

3. Selección de la muestra (muestreo)

Indique tamaño de muestra, método y justificación.

El programa no extrae una única muestra, sino que genera dos muestras de diferentes proporciones a partir de la población base de 300 alumnos. Esto permite realizar un ejercicio comparativo:

- Muestra A (Representativa): Se definió en un 50% de la población total, lo que equivale a un tamaño de muestra de 150 alumnos
- Muestra B (Reducida): Se definió en un 3% de la población total, lo que equivale a un tamaño de muestra de 9 alumnos

Se utiliza el método de muestreo aleatorio simple. El objetivo del programa no es solo sacar un promedio, sino hacer un experimento: comparar qué pasa cuando recolectamos mucha información versus cuando recolectamos muy poca.

4. Estadística descriptiva

Presenten media, mediana, moda, rango, varianza, desviación estándar.

```

Resultados:
Media: 7.4
Mediana: 7.33
Moda: 5.83
Rango: 4.97
Varianza: 2.21
Desviación estándar: 1.49

```

para 150 alumnos

```

Resultados:
Media: 7.18
Mediana: 7.5
Moda: No hay
Rango: 3.8
Varianza: 1.65
Desviación estándar: 1.28

```

para 9 alumnos

5. Representación gráfica de los datos

Describan la gráfica elegida y su relevancia.

```

HISTOGRAMA DE FRECUENCIAS:
5 - 6 : *****
6 - 7 : *****
7 - 8 : *****
8 - 9 : *****
9 - 10 : *****

```

para 150

```

HISTOGRAMA DE FRECUENCIAS:
5 - 6 : ***
6 - 7 : *
7 - 8 : ***
8 - 9 : *
9 - 10 : *

```

para 9

Se utilizó el histograma con asterisco para señalar los valores encontrados de promedios

6. Interpretación de resultados

Expliquen qué indican los resultados y conclusiones.

¿qué indican los resultados?

Los resultados al ser aleatorios no salen siempre los mismos, así que no se puede llegar a una conclusión general, pero el programa genera los resultados y puede arrojar que el promedio es más alto o más bajo gracias a la aleatorización

Valor de 15% total